

השאלות האלו מצוינות, והן בדיוק סוג השאלות שמתרגלים שואלים ב"הגנת חובה" כדי לוודא שבאמת הבנת את הקונספטים מאחורי הקוד.

הנה השאלות מתורגמות לעברית, יחד עם תשובות מפורטות המבוססות על המימוש שבנינו:

1. מה ההבדל בין ריבוי משימות שיתופי (Cooperative) לריבוי משימות מופקע (Preemptive)? כיצד המימוש שלנו (co_yield) מממש ריבוי משימות שיתופי?

- **ריבוי משימות מופקע (Preemptive):** מערכת ההפעלה (המתזמן) היא הבוסית. היא משתמשת בפסיקות חומרה (כמו טיימר) כדי להפסיק תהליך בכוח גם אם הוא באמצע חישוב, ומעבירה את המעבד לתהליך אחר. התהליכים לא שולטים מתי הם מופסקים.
- **ריבוי משימות שיתופי (Cooperative):** התהליכים הם אלה שמחליטים מתי לשחרר את המעבד. תהליך ירוץ עד שהוא יחליט מרצונו החופשי לוותר על השליטה (למשל, כשהוא מסיים משימה או מחכה למשהו).
- **איך co_yield קשור?** המימוש שלנו הוא דוגמה קלאסית למודל שיתופי. התהליכים (האבא והילד) מעבירים את המעבד אחד לשני באופן אקטיבי ורצוני על ידי קריאה ל-co_yield, ולא מחכים שהטיימר של ה-xv6 יפסיק אותם בכוח.

2. כיצד מנגנון ה-co_yield עוקף את המתזמן (Scheduler) הרגיל? מהם היתרונות והחסרונות של גישה זו?

- **כיצד הוא עוקף:** במקום להפוך את תהליך היעד ל-RUNNABLE ולהחזיר את השליטה למתזמן (בעזרת sched()), המנגנון שלנו משנה את מצב היעד ישירות ל-RUNNING, מעדכן את מצביע המעבד (proc->c-target =), וקורא ל-swtch באופן ישיר מהקונטקסט של התהליך הנוכחי לקונטקסט של תהליך היעד.
- **יתרונות:** ביצועים מהירים מאוד. החלפת הקשר ישירה חוסכת את התקורה של החלפה אל חוט המתזמן (Scheduler thread) וממנו. בנוסף, זה נותן שליטה דטרמיניסטית (אנחנו יודעים בדיוק מי ירוץ הבא).
- **חסרונות:** פגיעה בהוגנות של המערכת (Fairness). אם שני תהליכים "מתמסרים" ביניהם לנצח מבלי להשתמש במתזמן, הם יכולים "להרעב" (Starvation) תהליכים אחרים שממתינים למעבד, כי המתזמן לא מקבל הזדמנות להתערב.

3. כיצד מועבר הערך (integer) בין התהליכים במהלך ה-co_yield? היכן הוא נשמר וכיצד הוא מוחזר?

- **היכן הוא נשמר:** הערך מועבר ישירות לתוך אוגר ה-a0 בתוך ה-trapframe של תהליך היעד (target->trapframe->a0 = value).
- **כיצד הוא מאוחזר:** בארכיטקטורת RISC-V (שעליה רץ xv6), האוגר a0 משמש לאחסון ערך החזרה של פונקציות וקריאות מערכת. כשתהליך היעד מתעורר וחוזר למרחב המשתמש (User Space), החומרה והקרנל קוראים את מה שיש ב-a0 ומחזירים אותו כתוצאה של הפונקציה co_yield. אצלנו, אנחנו פשוט "רומסים" את מה שהיה שם ושולטים את הערך שרצינו.

4. באילו תרחישים בעולם האמיתי מנגנון קורוטין (Coroutine) עשוי להיות שימושי? מהן דוגמאות למערכות שמשתמשות בריבוי משימות שיתופי?

- **תרחישים שימושיים:** מודל זה מעולה לשרתי אינטרנט שמטפלים באלפי חיבורים מקבילים (I/O Bound), שם המתנה למידע מהרשת יכולה פשוט להעביר את השליטה ללקוח הבא ביעילות בלי התקורה של

החלפת תהליכים כבדה בקרנל. זה שימושי גם במנועי משחקים (לעדכון ישויות) ומערכות זמן-אמת שבהן סדר הפעולות קריטי.

- **דוגמאות בעולם האמיתי:**

- סביבת **Node.js** וה-Event Loop שלה.
- שפות כמו **Go** (עם Goroutines) או **Python** (עם async/await).
- מערכות הפעלה ישנות כמו Windows 3.1 או Mac OS קלאסי היו מבוססות לחלוטין על מודל שיתופי.

5. במה שונה החלפת תהליכים ישירה (Direct Switch) מזו המופעלת על ידי המתזמן הרגיל?

- **החלפה רגילה:** דורשת **שתי** החלפות הקשר (Context Switches). תהליך א' קורא ל-switch כדי לעבור לחוט של המתזמן. המתזמן מחפש תהליך אחר, ואז קורא שוב ל-switch כדי לעבור לתהליך ב'.
- **החלפה ישירה שלנו:** דורשת **החלפה אחת בלבד**. הקוד שלנו מדלג על חוט המתזמן לגמרי ומשתמש ב-switch כדי לעבור היישר מתהליך א' לזיכרון והרגיסטרים של תהליך ב'.

6. מה היה קורה אילו ניסינו לממש את הפונקציונליות הזו תוך שימוש אך ורק בקריאות מערכת קיימות (כמו sleep ו-wakeup)? מה היה חסר?

- אם היינו משתמשים רק ב-sleep ו-wakeup, לא היינו יכולים להבטיח מעבר **מידי**. הפונקציה wakeup רק משנה את הסטטוס של תהליך מ-SLEEPING ל-RUNNABLE. היא לא מריצה אותו. המתזמן הוא זה שפעם ב... יחליט להריץ אותו.
- בנוסף, היה חסר לנו מנגנון **להעברת הערך**. היינו צריכים להשתמש במשאבים מורכבים ואיטיים כמו צינורות (Pipes) או זיכרון משותף, במקום הזרקה ישירה ומהירה לרגיסטר.

7. כיצד המימוש שלך ב-co_yield משתלב עם מצבי התהליך הקיימים ב-xv6 (כגון RUNNING, RUNNABLE, SLEEPING)?

- המימוש משתמש במצבים הקיימים אבל "משחק" עם המעברים ביניהם:
- התהליך הקורא מעביר את עצמו ל-SLEEPING ומגדיר את ערוץ ההמתנה (chan) להיות תהליך היעד.
- המימוש **מדלג** על המצב RUNNABLE לחלוטין עבור תהליך היעד, ומעביר אותו ישירות ל-RUNNING.
- המימוש גם בודק את המצבים כמנגנון הגנה: הוא יאפשר מעבר רק אם היעד הוא RUNNABLE (מוכן לרוץ) או SLEEPING (כבר מחכה לנו ספציפית), כפי שהוגדר בתנאי שכתבת בקוד.

8. מהן ההשלכות על הביצועים של עקיפת המתזמן? באילו מצבים הדבר עשוי להיות מועיל או מזיק?

- **מועיל:** ביצועים מקסימליים כשיש צימוד חזק (Tight Coupling) בין שני תהליכים, כמו מודל של "צרך-צרך" (Producer-Consumer) שמתמסרים במידע כל רגע. עקיפת המתזמן חוסכת את מחזור החיפוש (Scheduler Loop) ואת ההחלפה הכפולה, מה שמוזיל משמעותית את עלות המעבר.
- **מזיק:** במערכות עם עומס גבוה והרבה משתמשים. מכיוון שהתהליכים שעוקפים את המתזמן לא מושפעים ממדיניות של תעדוף (Priority) או הקצאת זמן (Time Slices), קבוצה של קורטינות יכולה פשוט

להשתלט על מעבד שלם (CPU Hogging) ולהקפיא תהליכים אחרים (כמו תהליך של הדפסה למסך או קלטת הקלדות).